

A Gentle Introduction to Python

Teshome K. Deressa^{a,b,c}

^aGuest Research Fellow
Centre for Environmental Sciences
[Hasselt University, Belgium](#)

^bPostdoctoral Researcher
Centre for Neuroscience & Cell Biology
[University of Coimbra, Portugal](#)

^cData Analyst – Freelancer
Framework for **O**pen & **R**e reproducible **R**esearch **T**raining – **FORRT**
[Remote, Germany](#)

- ① What is & why Python?
- ② Installing Anaconda
- ③ Data types & structures
- ④ String manipulation ⚠
- ⑤ Working with NumPy
- ⑥ Data wrangling with pandas
- ⑦ Data visualization: matplotlib and seaborn
- ⑧ Basic statistical analysis: SciPy and scikit-learn
 - Independent t -test & simple linear regression

Lunch Break



What is Python and why?

What is Python?

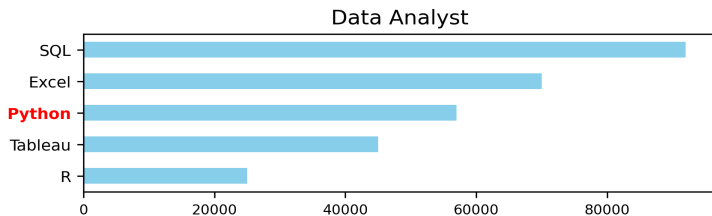
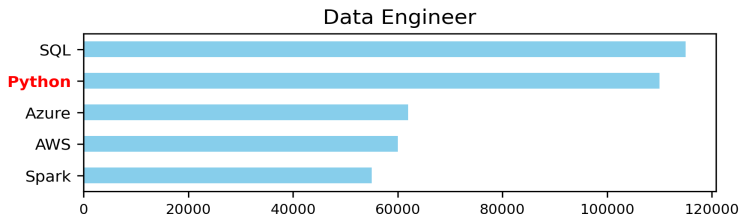
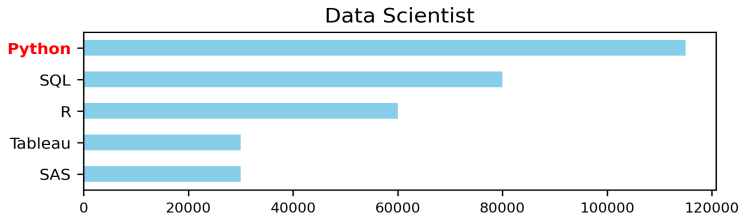
- High-level programming language

Why Python?

- Easy to learn
- Open source
- Community support
- High industry demand
- ⋮



Top 5 skills in job postings



I hope you already downloaded Anaconda! 🙌

<https://www.anaconda.com/download/success>

Awesome!

So, where is your Jupyter Notebook?

Probably, you're one click behind 😊

Anaconda

- *“Free, open-source software distribution for Python and R that simplifies data science, ML, and AI by bundling Python preprogramming languages”*



MACHINE LEARNING PYTHON PACKAGES



Data types

- Integer (numeric characters)
- Float (numerical characters with decimals)
- String (numerics and strings)
- Boolean (True/False)

Type	Example	Command
integer	x = 22	type(x)
float	y = 10.5	type(y)
text	z1 = 'Hello World!'	type(z1)
	z2 = "15.0"	type(z2)
boolean	3 == 5	False
	3 <= 5	True

- Data type conversion can be done using `int()` `float()`, `str()`



- list
- tuple
- dict

List

- Created using square brackets []
- Store both objects of the same & different types
- Mutable so their content can be modified in-place



List	Type
<code>lst1 = [1, 2, 3, 4, 5]</code>	numeric
<code>lst2 = ['hi', 'hello', 'mama']</code>	text
<code>lst3 = ['hi', 'hello', 2, 'mama']</code>	mixed

Note!

- In Python, **sequences are indexed starting from 0**
 - accessing the 1st element of `lst1` use `lst1[0]`
- Negative index accesses items from the end of the list—counting backwards
 - `lst1[-1]`, `lst1[-2]`
- Value of a specific item can be changed by referring to the index number
 - `lst1[-2] = 9`
- Range of indexes can be selected by using two arguments between brackets
 - `lst3[1:3]` # index 3 not included



Activity	Code
Adding element	<code>lst1.append(6)</code>
Inserting element in specific location	<code>lst1.insert(0, 6)</code>
Removing element by value	<code>lst1.remove(2)</code>
Selecting element by index	<code>lst1.pop(2)</code>
Sorting	<code>lst1.sort()</code>
Reversing	<code>lst1.reverse()</code>
Concatenating	<code>> lst1 + lst2</code> <code>> lst1.extend(['2', '10', '20'])</code>



In the interest of time ... reading assignment! 🙌



String manipulation (1) ⚠

```
1 print("Hello Oromia!")
```

- Python is great at dealing with string manipulation
- String $\equiv$ textual data
 - ▷ `office = "Oromia, Health, Bureau"`
- Python recognizes any sequence of characters inside quotes as a single string object
- Check: `len(office)`
- We can access a string in the same way as you did with lists
 - ▷ `office[3]` #the letter "m" of the word "Oromia"



- To take the entire value “Oromia”:
 - `office[0:6]`
- Check if a string is part of a bigger string:
 - `"health" in office`
- Converting to uppercase or lowercase:
 - `office.upper()`
 - `office.lower()`
- Splitting into pieces:
 - `office.split()`
 - `office.split(",")` → by any character
- Replacing:
 - `office.replace(", ", " ")`



Working with NumPy

- *NumPy* $\equiv$ Numerical Python
- Foundational library for numerical computing
- Store data in a structured format, making it easier to organize, access, and manipulate
- Import the *NumPy* package using

```
1 import numpy as np
```



Note!

- Use the `np.array([ ])` function to create Numpy arrays
- Basic operations: `+`, `-`, `/`, `*` performed **element-wise**

```
1  import numpy as np
2
3  x = np.array([1, 2, 3, 4, 5]) # creation
4
5  x_plus_one = x + 1
6  x_minus_one = x - 1
7  x_half = x / 2
8  x_double = 2 * x
9  x_sum = np.sum(x)
10 x_prod = np.prod(x)
11 x_cumsum = np.cumsum(x)
12 x[1:3] # slicing & watchout the indices!
```




```
1 import numpy as np
2
3 x_mean = np.mean(x)
4 x_median = np.median(x)
5 x_var = np.var(x)
6 x_std = np.std(x)
7 x_min = np.min(x)
8 x_max = np.max(x)
```

NaNs

- Generate NaN values using `np.nan`
- `np.nan` value is used in *NumPy* (and by extension in *pandas*) to represent missing or undefined data

```
1 y = np.array([10, 15, np.nan, 9, 30, np.nan, 8])
```



Data wrangling with pandas

- Popular open-source library for data preparation, manipulation & analysis
- Comes from the term “panel data”
- Built on top of NumPy
- Provides data structures like *Series* (1D-labeled array) & *DataFrame* (2D-data)
- Import the pandas package using

```
1 import pandas as pd
```



- Process of reading and loading data from various sources
- Two important properties:
 - **Format**
 - various formats: .csv, .xlsx, .sas7bdat, .sav, .dta, .mat, etc
 - **File path**
 - Computer: /Desktop/PyData/file_name.csv
 - Website: <https://github.com/teshomey/Data/blob/main/tips.csv>



Importing a CSV (2)

```
1
2 import pandas as pd  # import pandas library
3
4 file_path = 'C:/Users/user/Desktop/PyData/rainfall.csv' # path + name
5 df = pd.read_csv(file_path) # assign the DataFrame to variable df
6
7 df.head() # check the first 5 rows of the df
8 df.head(n) # choose any n
9
10 df1 = pd.read_csv(file_path, header=None) # importing without header
11 df1.head()
12
13 headers = ['station_code', 'city_name', 'year', 'month', 'day',
14            'rainfall', 'period', 'quality', 'lat', 'long', 'station_name',
15            'avg_rainfall']
16 df1.columns = headers # adding or replacing default headers
```



Importing other file types (3)

```
1 import pandas as pd
2
3
4 file_path = '....'
5
6 pd.read_excel('file_path') # Excel .xlsx data
7 pd.read_sas('file_path')   # SAS data
8 pd.read_stata('file_path') # Stata data
9 pd.read_spss('file_path')  # SPSS data
```



- Preserve progress anytime by saving modified data
- Export data using

```
1 df.to_csv('filename.csv', index=False)
2
3 df.to_excel('filename.xlsx', index=False)
4
5 df.to_stata('filename.dta', write_index=False)
```

Creating DataFrame

- We can also create a DataFrame using the pandas

```
1 import pandas as pd
2 data = {
3     'age': [20, 53, 44, 37, 26, 41, 39],
4     'height': [176, 167, 170, 173, 170, 165, 174],
5     'weight': [77, 56, 80, 89, 71, 62, 75],
6     'chol': [195, 250, 304, 178, 206, 284, 232],
7     'smoke': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes', 'Yes'],
8     'blood': ['B', 'O', 'A', 'O', 'O', 'O', 'O'],
9     'mort': ['Alive', 'Dead', 'Dead', 'Alive', 'Alive', 'Alive', 'Alive']
10 }
11
12 df = pd.DataFrame(data)
13 print(df)
14
15 df.index = ['patient 1', 'patient 2', 'patient 3', 'patient 4',
16            'patient 5', 'patient 6', 'patient 7'] # changing the index
17 print(df)
```



Filtering (6)

```
1 df[df['blood']=='O'] # filtering patients with blood type 'O'
2
3 df[df['age'] > 40]
4
5 df[(df['blood']=='O') & (df['mort']=='Alive')]
6
7 df[(df['smoke']=='Yes') | (df['mort']=='Dead')]
8
```

Note!

We can select column of a DataFrame in two ways:

- 1 `df['col_name']` → Personally, I prefer this one!
- 2 `df.col_name`



Dealing with NaNs (7-1)

- Missing values are very common
- In Python, missing values are represented by NaN → “Not a Number”

```
1 import pandas as pd
2 import numpy as np # we need NumPy for creating NaNs
3
4 country = {
5     'country': ['Belgium', 'France', 'Italy', 'Spain'],
6     'area': [30.689, 643.801, 301.338, 505.990],
7     'prison_pop': [11.071, 71.190, 54.653, 61.526],
8     'population': [11.31, 66.60, 60.67, 44.44],
9     'life_exp': [80.99, np.nan, 82.54, np.nan] # special np.nan
10                                             # object used to represent missing
11 }
12
13 df = pd.DataFrame(country)
14 print(df)
```



Dealing with NaNs (7-2)

```
1
2 df.isnull()           # detect NaNs
3
4 df.notnull()          # detect non-NaN
5
6 df.isnull().any()      # which column contains NaNs?
7
8 df.dropna(axis = 0)    # remove ROWS with NaNs
9
10 df.dropna(axis = 1)   # remove COLUMNS with NaNs
11
12 df.fillna(0)          # replacing NaNs by 0 - NOT recommended!
13
14 df.fillna(df['life_exp'].mean()) # replace NaNs by mean
15
```



Merging (8)

- Combining DataFrames along specific column or multiple columns

INNER JOIN

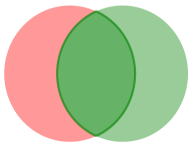


Table A Table B

LEFT JOIN



Table A Table B

RIGHT JOIN

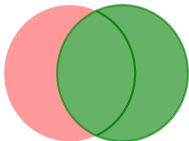


Table A Table B

OUTER JOIN



Table A Table B

Note!

- INNER JOIN → Only the overlapping part
- LEFT JOIN → All of Table A + overlap
- RIGHT JOIN → All of Table B + overlap
- OUTER JOIN → Everything from both tables



Merging (9)

```
1 pd.merge(df1, df2, on=, how=)
```

```
1 import pandas as pd
2
3 df1 = pd.read_csv('C:/Users/user/Desktop/PyData/alcohol_2005.csv')
4 df2 = pd.read_csv('C:/Users/user/Desktop/PyData/alcohol_2008.csv')
5
6 pd.merge(df1, df2) # problem! why?
7
8 pd.merge(df1, df2, on='code') # inner join based on one column
9 pd.merge(df1, df2, on=['code', 'country']) # based on multiple columns
10 pd.merge(df1, df2, on=['code', 'country'], how='left') # left join
11 pd.merge(df1, df2, on=['code', 'country'], how='right') # right join
12 pd.merge(df1, df2, on=['code', 'country'], how='outer') # outer join
```



Inspection (10)

```
1 tips = pd.read_csv('C:/Users/user/Desktop/PyData/tips.csv')
2
3 tips.info()      # getting a quick glance at the dataset
4 tips.count()     # number of non-NaNs
5
6 tips.head()
7 tips.head(n)
8 tips.tail()
9
10 len(tips)        # length of DataFrame
11 tips.shape       # dimensions (rows, columns)
12 tips.dtypes      # data type of each column
13
14 tips.columns     # describe DataFrame columns
15 tips['sex']       # accessing a column
16 tips[['sex', 'smoker']] # accessing columns - use double square brackets
17 tips['sex'].unique() # number of distinct values in a column
```



Descriptive statistics (11)

```
1 tips.describe() # summary statistics of numerical columns
2 tips['tot_bill'].describe() # specific column
3
4 tips.sum()      # sum of values
5 tips.cumsum()   # cumulative sum of values
6 tips.min()      # min. value
7 tips.max()      # max. value
8 tips.mean()
9 tips.median()
10 tips.mode()
11 tips['smoker'].value_counts() # frequency table - I love this :-)
12
13 tips.corr()
14 tips.corr(method='spearman')
15 tips.corr(method='kendall')
```



Grouping (12)

```
1 import pandas as pd
2 tips = pd.read_csv('C:/Users/user/Desktop/PyData/tips.csv')
3 tips.head()
```

Example

- 1 Get the sum of total bill by sex
- 2 Get the sum of total bill & tips by sex
- 3 Get the sum of total bill, tips & size by sex and smoking behaviour
- 4 Get average tip for lunch & dinner time

```
1 tips.groupby('sex')['tot_bill'].sum() # Q1
2 tips.groupby('sex')[['tot_bill', 'tip']].sum() # Q2
3 tips.groupby(['sex', 'smoker'])['tot_bill', 'tip'].sum() # Q3
4 tips.groupby('sex')['tip'].mean() # Q4
5 tips.groupby('sex')['tot_bill', 'tip'].agg(['min', 'max']) # Bonus!
```



Data visualization: matplotlib and seaborn

- **matplotlib** makes it easy to create meaningful plots
- Let's try! 😊

```
1 import matplotlib.pyplot as plt  # pyplot is subpackage of matplotlib
2
3 year = [2000, 2004, 2008, 2012, 2016]
4 pris_pop = [8.688, 9.245, 9.858, 11.212, 11.071]
5
6 plt.plot(year, pris_pop) # used for line plots
7
8 plt.plot(year, pris_pop, marker= "*")
9
10 plt.scatter(year, pris_pop) # used for scatter plots
11 plt.xlabel('Year')
12 plt.ylabel('Prison Population')
13 plt.title('Prison Population by Year')
14 plt.show()
```



```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 tips = pd.read_csv('C:/Users/user/Desktop/PyData/tips.csv')
5
6 plt.hist(tips['tot_bill'])           # produce histogram
7 plt.hist(tips['tot_bill'], bins=5)  # adding bins
8 plt.boxplot(tips['tot_bill'])       # boxplot
9
10 plt.savefig('my_histogram.png')    # save the plot
11                                   # DON'T forget specifying the directory!
12 plt.show()
13
```

seaborn: scatter plots (1)

- seaborn is built on top of matplotlib
- Advanced statistical visualizations

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns          # sns is used as alias for seaborn
3
4 tips = pd.read_csv('C:/Users/user/Desktop/PyData/tips.csv')
5
6 sns.scatterplot(x='tot_bill', y='tip', data=tips)
7 plt.xlabel('Total bill')
8 plt.ylabel('Tip')
9 plt.show()
10
11 sns.scatterplot(x='tot_bill', y='tip', hue='sex', data=tips) # subgroup
12 plt.xlabel('Total bill')
13 plt.ylabel('Tip')
14 plt.show()
```



- Creating separate scatterplots for **subgroup** – `sns.relplot()`

```
1 myplot = sns.relplot(x='tot_bill', y='tip', kind='scatter',  
2                      col='sex', data=tips)  
3 myplot.set_xlabels('Total bill', 'Total bill')  
4 myplot.set_ylabels('Tip', 'Tip')  
5 plt.show()  
6  
7 # subplot for each combination of time & sex  
8 sns.relplot(x='tot_bill', y='tip', kind='scatter',  
9            row='time', col='sex', data=tips) # watchout the row= & col=  
10
```

```
1 import pandas as pd
2
3 df = pd.read_csv('C:/Users/user/Desktop/PyData/rainfall.csv')
4 df.columns
5
6 myplot = sns.relplot(x='year', y='avg_rainfall', kind='line', data=df)
7 myplot.set_xlabels('Year')
8 myplot.set_ylabels('Average Rainfall')
9 plt.show()
10
11 # line plot for each city
12 sns.relplot(x='year', y='avg_rainfall', kind='line',
13             style='city_name', hue='city_name', data=df)
14 plt.xlabel('Year')
15 plt.ylabel('Average Rainfall')
16 plt.show()
```

seaborn: bar plots (4)

```
1
2 # count plot
3 sns.catplot(x='day', kind='count', data=tips)
4
5 # bar plot (mean of tot_bill by each day)
6 sns.catplot(x='day', y='tot_bill', kind='bar', data=tips)
7
8 # box plot
9 sns.catplot(x='day', y='tot_bill', kind='box', data=tips)
10
11 # swarm plot
12 sns.swarmplot(x='day', y='tot_bill', data=tips)
```



Basic statistical analysis

```
1 import pandas as pd
2 df = pd.read_excel('C:/Users/user/Desktop/PyData/bp_reduction.xlsx')
3 df.head()
4
5 df['group'].value_counts()
6 df.groupby('group')['bp_reduc'].describe()
```

Example

- There seems to be a difference between the mean values of two drugs
- Is this difference statistically significant?



Independent t -test – Lab dataset (2)

```
1 drugA = df[df['group']=='Drug A'] # Create 2 DataFrames
2 drugB = df[df['group']=='Drug B']
3
4 # Levene's test (check homogeneity of variance)
5 from scipy import stats # import stats functions from the scipy package
6 stats.levene(drugA['bp_reduc'], drugB['bp_reduc'])
7                                     # check the p-value
8                                     # equal or unequal var?
9 # independent t-test
10 stats.ttest_ind(drugA['bp_reduc'], drugB['bp_reduc'],
11                 alternative='two-sided', equal_var=True) # conclusion?
```

Note!

- `alternative='less'` → tests if $\text{mean}(A) < \text{mean}(B)$
- `alternative='greater'` → tests if $\text{mean}(A) > \text{mean}(B)$
- `stats.ttest_rel()` → paired t -test



Independent t -test – Iris dataset (1)

```
1 import pandas as pd
2
3 iris = pd.read_csv('C:/Users/user/Desktop/PyData/iris.csv')
4 iris.head()
5 iris['species'].value_counts()
6
7 # focus on species & sepal_width
8 iris.groupby('specie')['sepal_width'].describe()
```

Example

- There seems to be a difference between the average sepal width for setosa & versicolor
- Is this difference statistically significant?



Independent t -test – Iris dataset (2)

```
1 # Create 2 DataFrames
2 setosa = iris[iris['species'] == 'Iris-setosa']
3 versicolor = iris[iris['species'] == 'Iris-versicolor']
4
5 # Levene's test
6 from scipy import stats
7 stats.levene(setosa['sepal_width'], versicolor['sepal_width'])
8
9 # independent t-test
10 stats.ttest_ind(setosa['sepal_width'], versicolor['sepal_width'])
```

Conclusion

- Statistically significant difference between the sepal width of setosa and versicolor ($t(98) = 9.282, p < 0.0001$)



Example 1: Simple linear regression (1)

Example

- Consider `life_expectancy.csv` dataset
- **Target:** Life expectancy
- **Health indicators:** Adult Mortality, Infant deaths, BMI, Hepatitis B, Measles, etc
- **Economic & social indicators:** GDP, Schooling, Population, Total expenditure, etc
- **Objective:** Does GDP predict life expectancy?

① Import required libraries

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.model_selection import train_test_split
4 from sklearn.linear_model import LinearRegression
5 from sklearn.metrics import mean_squared_error, r2_score
```



Example 1: Simple linear regression (2)

② Load the dataset and explore

```
1 df = pd.read_csv('C:/Users/user/Desktop/PyData/life_expectancy.csv')
2 df.head()
3 df.describe()
4
5 plt.scatter(df['Life expectancy'], df['GDP'], color='blue')
6 plt.xlabel('Life expectancy')
7 plt.ylabel('GDP')
8 plt.show()
```

③ Choose feature(s) and target variable

```
1 X = df[['GDP']]    # feature & must be 2D
2 y = df['Life expectancy']
```



Example 1: Simple linear regression (3)

4 Split into train and test sets

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X, y, test_size=0.2, random_state=42)
```

5 Fit the model

```
1 model = LinearRegression()  
2 model.fit(X_train, y_train)  
3  
4 print('Intercept:', model.intercept_)           # intercept  
5 print('Coefficient for GDP:', model.coef_[0])    # GDP coefficient
```

6 Predict on **test data**

```
1 y_pred = model.predict(X_test)
```



Example 1: Simple linear regression (4)

7 Evaluate performance

```
1  # check the regression line
2  plt.scatter(X_test, y_test, color='blue', label='Actual')
3  plt.plot(X_test, y_pred, color='red', linewidth=2, label='Predicted line')
4  plt.xlabel('GDP')
5  plt.ylabel('Life expectancy')
6  plt.title('Life Expectancy vs GDP')
7  plt.legend()
8  plt.show()
9
10 mse = mean_squared_error(y_test, y_pred)  # MSE
11 r2 = r2_score(y_test, y_pred)             # R^2
12
13 print('Mean Squared Error:', mse)
14 print('R² Score:', r2)
```

Conclusion

- What's your conclusion? 🤔



Note!

- There are several possible Python packages that you can use to run a linear regression



Thanks. 🙏

Struggling with fitting simple and advanced statistical models? What about deploying machine learning algorithms? Wait ... which programming language do you use? Python? R? SAS? No worries, feel free to reach out to get quality service! 🔥

Contact

✉ tashe.zgreat@gmail.com 📞 +251 92 234 5086

🌐 [LinkedIn](#) 🐙 [GitHub](#) 📖 [Google Scholar](#)

Motto → Quality over quantity!

